

Unit 3 — Planning with Tools & the Coordinator-Worker-Delegator Model

FF, LLM-based & HTN planning • Integrating tools + planning • CrewAI / AutoGen / LangGraph • CWD model, roles, communication

Contents

1. Planning algorithms recap (A*, GraphPlan, MCTS)
2. Fast Forward (FF) planning + code
3. LLM-based planning (CrewAI example)
4. HTN planning (decomposition + CrewAI hierarchical)
5. Integrating tool use and planning
6. Reasoning about tools; planning for tool use
7. Frameworks: CrewAI, AutoGen, LangGraph + comparison
8. Chapter 5 Q&A
9. The CWD model — roles, principles
10. CWD for the intelligent travel agent
11. Designing agents with role assignments
12. Communication & collaboration between agents
13. Implementing CWD in generative-AI systems (intro)

1. Recap — Less Practical Planners for LLM Agents

Algorithm	Problem when used with LLM agents
A* planning	Needs a clear action-cost function <i>and</i> heuristic estimate of remaining cost; impossible to quantify the "distance" between conversation states or the "cost" of reaching an understanding.
GraphPlan	Layered action/effect graph breaks down: language isn't discrete layers with clear cause-and-effect; combinatorial explosion; mutual-exclusion between conversational actions hard to determine → intractable.
MCTS	Each simulation = real LLM calls (too costly); random sampling inefficient in the vast language space; game-like strength becomes weakness in open-ended language.

2. Moderately Practical — Fast Forward (FF) Planning

FF uses **heuristic search with a simplified (relaxed) planning problem** to guide search.

Advantages for LLM agents	Challenges
• Goal-oriented approach aligns with how LLMs complete tasks • Relaxed planning gives useful approximations for complex language tasks • Heuristic guidance manages huge language search space • Flexible: works with partial state descriptions — valuable in natural language	• Original numeric heuristics don't translate to language states • Relaxed plans risk oversimplifying rich language context • Hard to define clear delete effects (what a conversation action removes/changes) • State representation needs substantial adaptation to natural language

```

class LLMFastForward:
    def create_relaxed_plan(self, current_state: str, goal: str) -> list:
        """Create a simplified plan ignoring complexities"""
        prompt = f"Given current state: {current_state}\nAnd goal: {goal}\n"
        prompt += "Generate a simplified step-by-step plan"
        return self.llm.generate_plan(prompt)

    def select_next_action(self, relaxed_plan: list):
        """Choose next action based on the relaxed plan"""
        return relaxed_plan[0]  # simplified selection

```

- `create_relaxed_plan` — takes current state & goal as text, asks the LLM for a high-level simplified plan (ignores complexities, like FF ignores delete effects).
- `select_next_action` — here simply takes the first action (`relaxed_plan[0]`); a real one would add selection logic.
- Verdict: **moderately practical** — possible but needs significant modification from its original form.

3. Most Practical — LLM-Based Planning & HTN

These align naturally with how language models process information and handle complex tasks; they embrace the fluid, contextual nature of language-based planning that traditional planners struggle with.

3.1 LLM-based planning

LLMs **understand complex goals, generate appropriate steps, and adapt them as context changes**. Unlike traditional planners needing explicit state representations, LLM planners work with **natural-language descriptions of states and actions** — flexible and expressive.

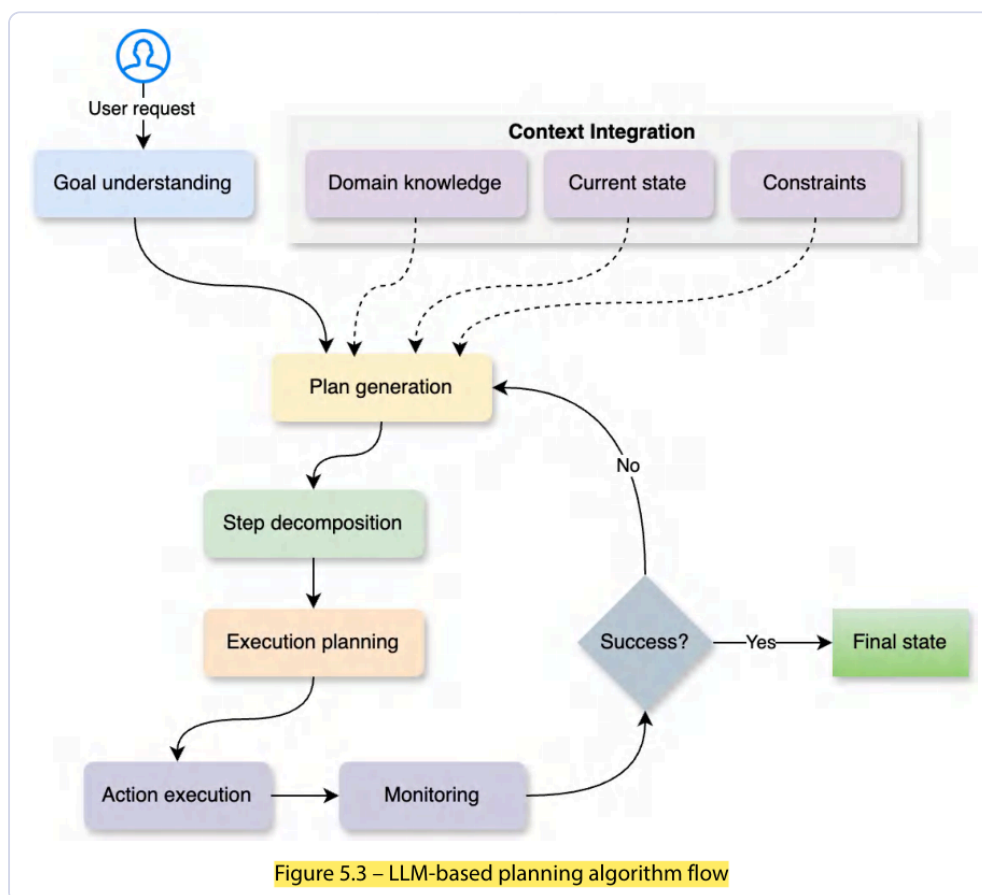


Figure 5.3 — LLM-based planning algorithm flow

User request → Goal understanding (+ domain knowledge, current state, constraints) → Plan generation
→ Step decomposition → Execution planning → Action execution → Monitoring → Success? (No → replan /
Yes → Final state)

CrewAI implementation — TravelPlanner Two specialised agents: **Travel Planning Strategist** (breaks requests into manageable steps) and **Travel Researcher** (validates and finds options). Two tasks (planning_task → research_task) run in a **Process.sequential** crew via `crew.kickoff(inputs={"request": request})`. The planning task: understand client requirements, specific booking requirements, required validations; research task: flight availability, hotel options, local transportation.

```
class TravelPlanner:
    def __init__(self):
        self.planner = Agent(role='Travel Planning Strategist',
                             goal='Create comprehensive, personalized travel plans', ...)
        self.researcher = Agent(role='Travel Researcher',
                                goal='Find and validate travel options and opportunities', ...)
    def create_travel_plan(self, request: str) -> Dict:
        planning_task = Task(description=f"Analyze the following travel request ... {request}",
                              agent=self.planner)
        research_task = Task(description="Based on the initial plan, research and validate: "
                                       "Flight availability, hotel options, and local transportation",
                              agent=self.researcher)
        crew = Crew(agents=[self.planner, self.researcher],
                    tasks=[planning_task, research_task], process=Process.sequential)
        return crew.kickoff(inputs={"request": request})
```

- **Advantages:** understands complex natural-language requests, generates steps dynamically, adapts to scenarios, agents share context and build on each other's output; handles nuance ("relaxing beach vacation with some cultural activities" → concrete recommendations); handles **novel situations** (STRIPS/A* need explicit states for every scenario); handles **uncertainty and partial information** by making reasonable assumptions and adding contingency steps.
- **Caveats:** may generate **overly optimistic or impractical plans** if not constrained; may struggle with **highly specific numerical constraints / strict timing** unless explicitly handled.

3.2 HTN (Hierarchical Task Network) planning

HTN Breaks complex tasks into simpler subtasks, forming a **hierarchy of actions**. Unlike STRIPS (primitive actions), HTN works with **abstract (compound) tasks** and decomposes them progressively until **primitive tasks** that can be executed directly.

```
def buy_groceries_task():
    return [('go_to_store', []), ('select_items', []), ('checkout', []), ('return_home', [])]

def select_items_task():
    return [('check_list', []), ('find_item', []), ('add_to_cart', [])]
```

`buy_groceries_task` is a high-level task split into four subtasks; `select_items` is further decomposed into three actions.

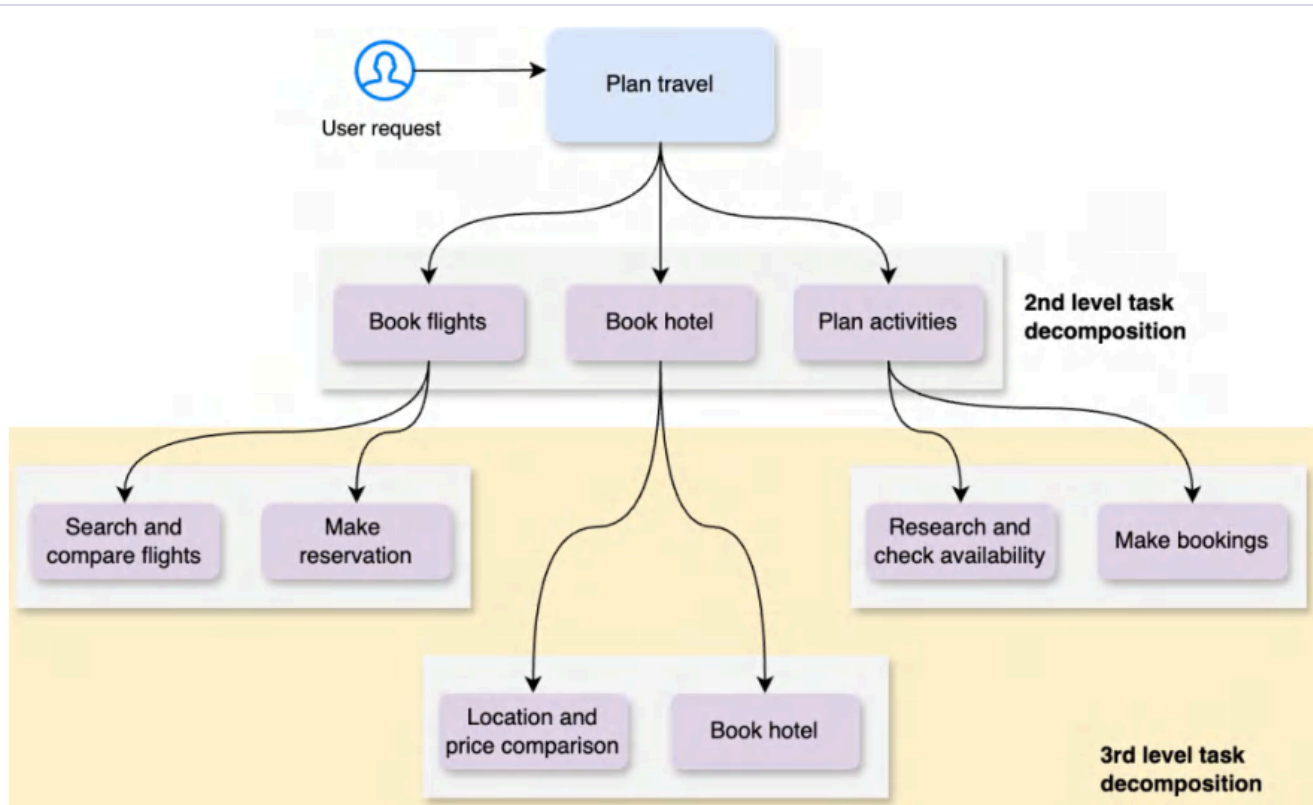


Figure 5.4 – HTN decomposition

Figure 5.4 — HTN decomposition: Plan travel → Book flights / Book hotel / Plan activities → 3rd-level tasks

- Travel decomposition: **Plan travel** → (2nd level) Book flights, Book hotel, Plan activities → (3rd level) Search & compare flights, Make reservation, Research & check availability, Make bookings, Location & price comparison, Book hotel.
- **CrewAI hierarchical process:** needs a **Manager** that breaks down tasks and delegates to agents. Manager can be an **agent** (control how tasks break down to n levels) or the **LLM itself** (`manager_llm`; arbitrary plan generated by LLM; prompt engineering gives some control but is less flexible — for simpler workflows).

```

flight_specialist = Agent(role='Flight Planning Specialist', goal='Handle all aspects of flight
arrangements', backstory='Expert in airline bookings and flight logistics.')
accommodation_specialist = Agent(role='Accommodation Specialist', goal='Manage all accommodation-related
planning', ...)
activity_specialist = Agent(role='Vacation Activity Specialist', goal='Manage all activity-related
planning', ...)
manager_llm = ChatOpenAI(model="gpt-4o-mini")
travel_planning_task = Task(description=f"Plan a comprehensive flight itinerary: {request} ...",
                             expected_output="A detailed flight itinerary ...", agent=None) # manager
delegates
crew = Crew(agents=[flight_specialist, accommodation_specialist, activity_specialist],
            tasks=[travel_planning_task], process=Process.hierarchical, manager_llm=manager_llm)
return crew.kickoff()
  
```

Important observation The sample output (5-day Paris trip for two adults from New York: flights \$2,960, hotel €800, transfers €100, Versailles day trip ≈€364.20) is **completely fictional** because the system has **no access to external tools/lookup** — this underscores why tools are needed.

Advantages of HTN: natural problem representation mirroring human thinking; better **scalability** (reduced search space); encodes **expert knowledge** in reusable task hierarchies; flexible with both abstract and primitive tasks (different levels of abstraction).

4. Integrating Tool Use and Planning

Earlier AI work treated planning algorithms and tool capabilities separately. Truly intelligent agents need them **integrated**: the travel planner gave a detailed but **non-factual** plan; to ground it in real flight/hotel/activity data, **tools must be combined with planning**.

4.1 Reasoning about tools

- Agents must understand each tool's **functionality, capabilities, limitations** and when/where it applies; assess tools against current goals and choose the most appropriate.
- For LLM agents this reasoning is **largely handled by the LLM**: provide clear tool descriptions; no complex reasoning code needed.

```
travel_agent = Agent(role='Travel Planner', goal='Plan comprehensive travel itineraries',
    tools=[flight_search_tool,      # finding and booking flights
           hotel_booking_tool,      # hotel reservations
           activity_planner_tool]) # local activities and attractions
```

The LLM agent naturally understands **which tool** to use per task, **when to combine** tools (coordinate flight & hotel dates) and **how to adapt** usage to requirements (budget).

Smaller / untrained models may	Even capable models may
Fail to understand when a tool is needed Make incorrect assumptions about capabilities Use tools in wrong sequences Miss opportunities to use tools Ignore tool constraints/requirements	Struggle with complex multi-step tool combinations Be inconsistent in tool selection across similar scenarios Confuse tools with subtle functional differences Struggle with error recovery when tools fail

Hence CrewAI, LangGraph and AutoGen work best with advanced models with strong tool reasoning, and tool-usage patterns must be **tested before deployment**.

4.2 Planning for tool use

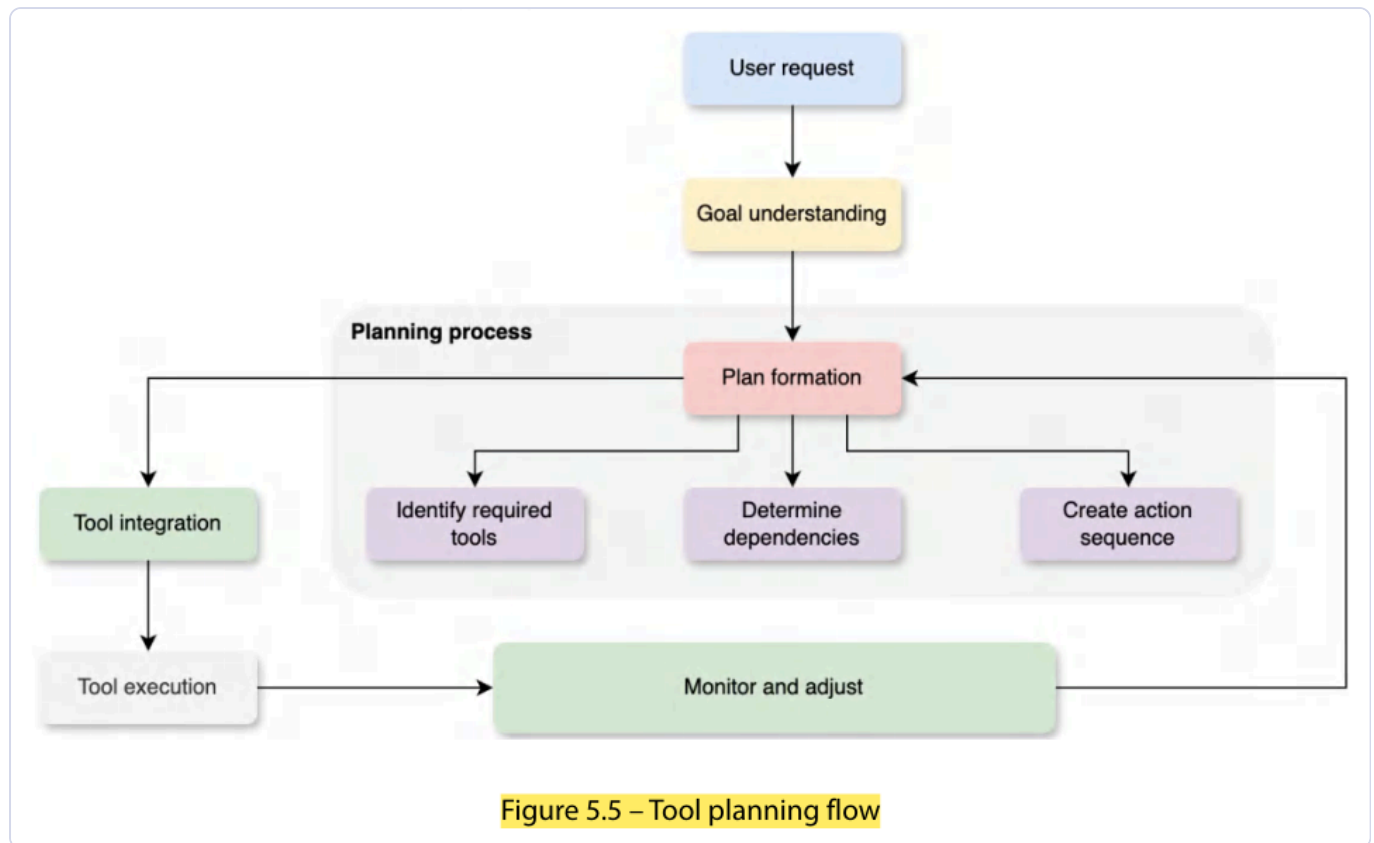


Figure 5.5 — Tool planning flow

1. **Goal understanding** via natural-language processing (e.g., a family vacation needs family-friendly accommodation and activities too).
2. **Plan formation**: identify required tools, determine dependencies, create action sequence — mirrors HTN decomposition but with LLM flexibility.
3. **Tool integration** — agent understands capabilities from descriptions, sequences them (confirm flight dates before hotels; activity planning considers location & timing).
4. **Tool execution**, then **monitor and adjust** based on new information/changed circumstances (like a human travel agent reacting to client feedback).

Combines the structure of traditional planning with the adaptability of language models.

5. Practical Implementations — Three Frameworks

5.1 CrewAI

- Provides a `@tool` decorator to define tools with descriptions/docs (`search_flights`, `find_hotels`, `find_activities`).
- Agent has **role**, **goal**, **backstory**, **tools**; Task has description, expected_output, agent.
- `crew.kickoff()`: understands requirements from task description → identifies tools needed from role & goals → uses tools in logical sequence → processes outputs into final response.

```
@tool("Search for available flights between cities")
def search_flights(...) -> dict:
    """Search for available flights between cities."""
agent = Agent(role='An expert travel concierge', goal='Handle all aspects of travel planning',
    backstory="Expert in airline bookings, hotel bookings, and booking vacation activities.",
    tools=[search_flights, find_hotels, find_activities], verbose=False)
```

5.2 AutoGen

- Platform where AI agents **converse** to solve tasks; uses `RoundRobinGroupChat` with four agents: **flight planner, hotel planner, activities planner, summary agent**.
- Each `AssistantAgent` has **name, description, model client** (e.g., GPT-4o-mini), **tools**, and **system_message**.
- Agent communication**: round-robin turn-taking (vs CrewAI's hierarchical task-based approach).
- Termination**: `TextMentionTermination` — summary agent ends conversation by saying `"TERMINATE"`. Parameters: `mention_text` (keyword), `case_sensitive`, `strip_whitespace`, `regex_match`.
- Tool integration**: tools associated directly with agents at initialisation (not decorators).
- Coordination pattern**: collaborative, agents contribute equally; summary agent creates the final integrated plan.

```
flight_agent = AssistantAgent(name="flight_planner", model_client=model_client,
    tools=[travel_tools.search_flights],
    description="A helpful assistant that can plan flights itinerary for vacation trips.",
    system_message="You are a helpful assistant that can plan flight itinerary ...")
group_chat = RoundRobinGroupChat([flight_agent, hotel_agent], termination_condition=termination)
await Console(group_chat.run_stream(task="I need to plan a trip to Paris from New York for 5 days."))
```

5.3 LangGraph (LangChain)

- Sub-framework of LangChain for LLM agent workflows; a **state machine / workflow graph** of nodes & edges (`@tool` decorator for tools).
- Two main nodes: **agent node** (processes messages, decides) and **tool node** (executes flight search, hotel booking, activity planning).
- Cycle: agent evaluates state → either calls tools or gives final response, controlled by a conditional function (`should_continue`) routing to tools or ending. Tools node **always returns to agent**. `MemorySaver` checkpoint persists state; graph compiled into a runnable.

```
workflow = StateGraph(MessagesState)
workflow.add_node("agent", call_model)
workflow.add_node("tools", tool_node)
workflow.add_edge(START, "agent")
workflow.add_conditional_edges("agent", should_continue)
workflow.add_edge("tools", 'agent')
checkpointer = MemorySaver()
app = workflow.compile(checkpointer=checkpointer)
final_state = app.invoke({"messages": [HumanMessage(content="I need to plan a trip to Paris from New York for 5 days")]},
    config={"configurable": {"thread_id": 42}})
```

Pros: explicit flow control (easy to visualise), built-in state management with **checkpointing**. **Cons:** needs graph-programming understanding; more initial setup than CrewAI.

Aspect	LangGraph	CrewAI	AutoGen
State management	Explicit state management	Via agent instances and task context	Via group-chat message history
Tool integration	Dedicated tool node	Decorator-based definition, direct agent association	Tools associated directly with specific agents
Flow control	Graph-based workflow	Hierarchical task decomposition or sequential flow	Round-robin turn-taking

6. Chapter 5 Questions & Answers

- 1. Purpose of tools; role of docstrings?** Tools are functions letting agents perform specific tasks/access external services; docstrings give purpose, expected parameters and return values so the LLM knows when/how to use them.
- 2. Traditional (STRIPS) vs LLM-based planning?** Traditional rely on explicit states & predefined actions with binary conditions; LLM-based works with natural language and handles ambiguous states — traditional can't represent nuanced contextual language tasks.
- 3. How does HTN work; why practical?** Progressive decomposition into simpler subtasks in a hierarchy; mirrors how LLMs process tasks; structured planning plus flexibility.
- 4. Reasoning in tool selection & its limits?** Largely the model's built-in ability; limits: inconsistent selection, complex combinations, error recovery.
- 5. Framework selection factors?** Workflow complexity (structured vs conversational), state management needs, multi-agent collaboration, development complexity. CrewAI = straightforward; AutoGen = multi-agent interaction; LangGraph = robust workflow control but more setup.

Exploring the Coordinator, Worker, and Delegator (CWD) Approach

7. The CWD Model

A framework for building **multi-agent systems** emphasising **collaboration, specialisation and effective task distribution/resource management**. Inspired by **organisational psychology and management theory** — clear role delegation and hierarchy as in human organisations. Suitable where autonomous agents collaborate on goals beyond a single agent's capability.

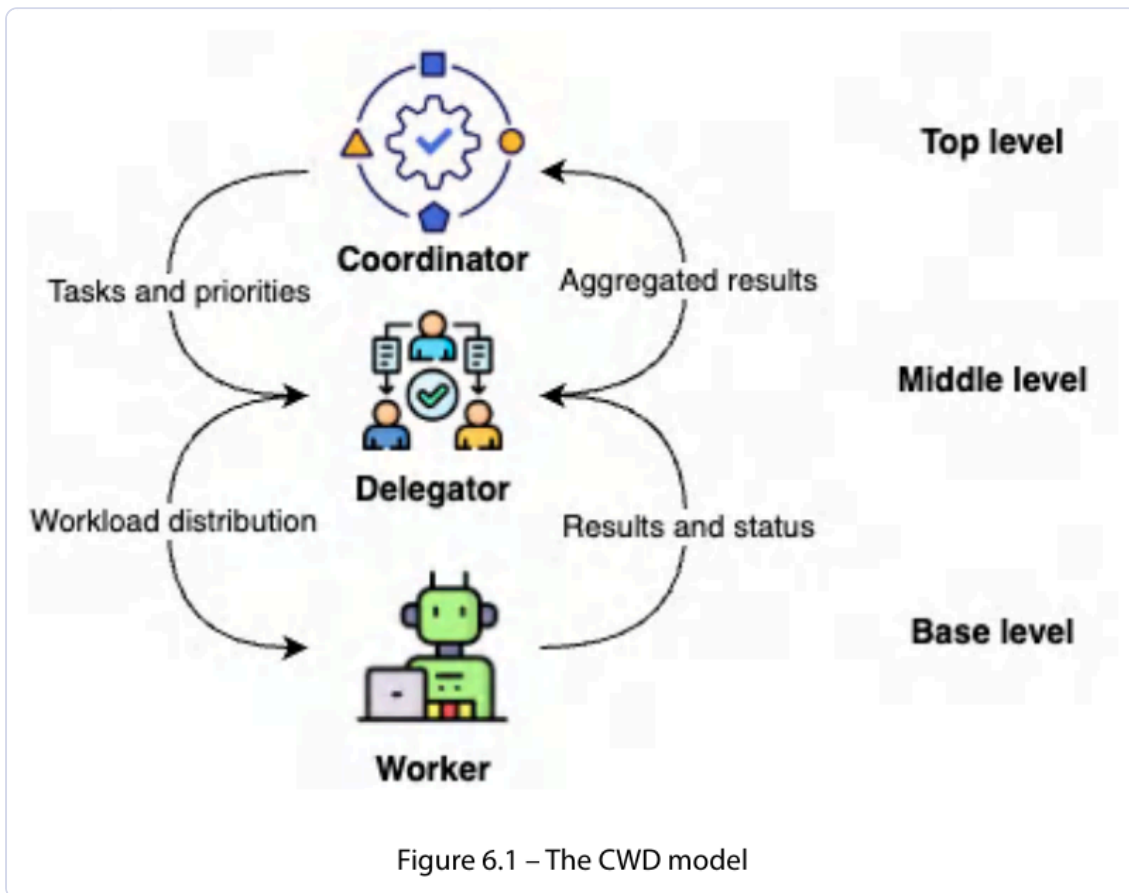


Figure 6.1 — The CWD model: Coordinator (top) ↔ Delegator (middle) ↔ Worker (base); tasks & priorities flow down, aggregated results flow up

Role	Responsibilities
Coordinator	Manages tasks, resources and overall workflow; monitors progress, assigns tasks to appropriate agents, enables collaboration among workers; orchestrator keeping the system aligned with objectives. Prioritises workloads by dynamically allocating tasks based on urgency, resource availability and dependencies; adjusts assignments.
Workers	Specialised agents carrying out specific tasks/functions; diverse capabilities and expertise; when assigned by delegators they use specialised knowledge to realise task objectives → division of labour, optimum overall performance.
Delegators	Intermediaries between coordinators and workers; implement workload assignments by resource availability and system needs; interface for communication; dispatch & balance workload; assign right tasks to right workers at the right time. Optimise throughput, latency and resource utilisation (min delays, max tasks per unit time, prevent bottlenecks).

7.1 Key principles of the CWD model

- **Separation of concerns** — strategic planning (coordinator), resource management (delegator), task execution (worker): each focuses on its core competency; flexibility and scalability.
- **Hierarchical organisation** — Top: strategic oversight & planning; Middle: resource management & coordination; Base: specialised task execution.
- **Information flow & feedback loops** — bidirectional: **downward** (task assignments, priorities, constraints), **upward** (progress updates, results, resource utilisation).
- **Adaptability & resilience:**
 - **Dynamic resource allocation** — reassess workload and redistribute resources in real time.
 - **Fault tolerance through redundancy** — overlapping capabilities allow seamless handoff and recovery.
 - **Load balancing** — distribute by availability, expertise, current workload.
 - **Runtime role reassignment** — agents adapt roles to evolving needs.

8. CWD for the Intelligent Travel Agent

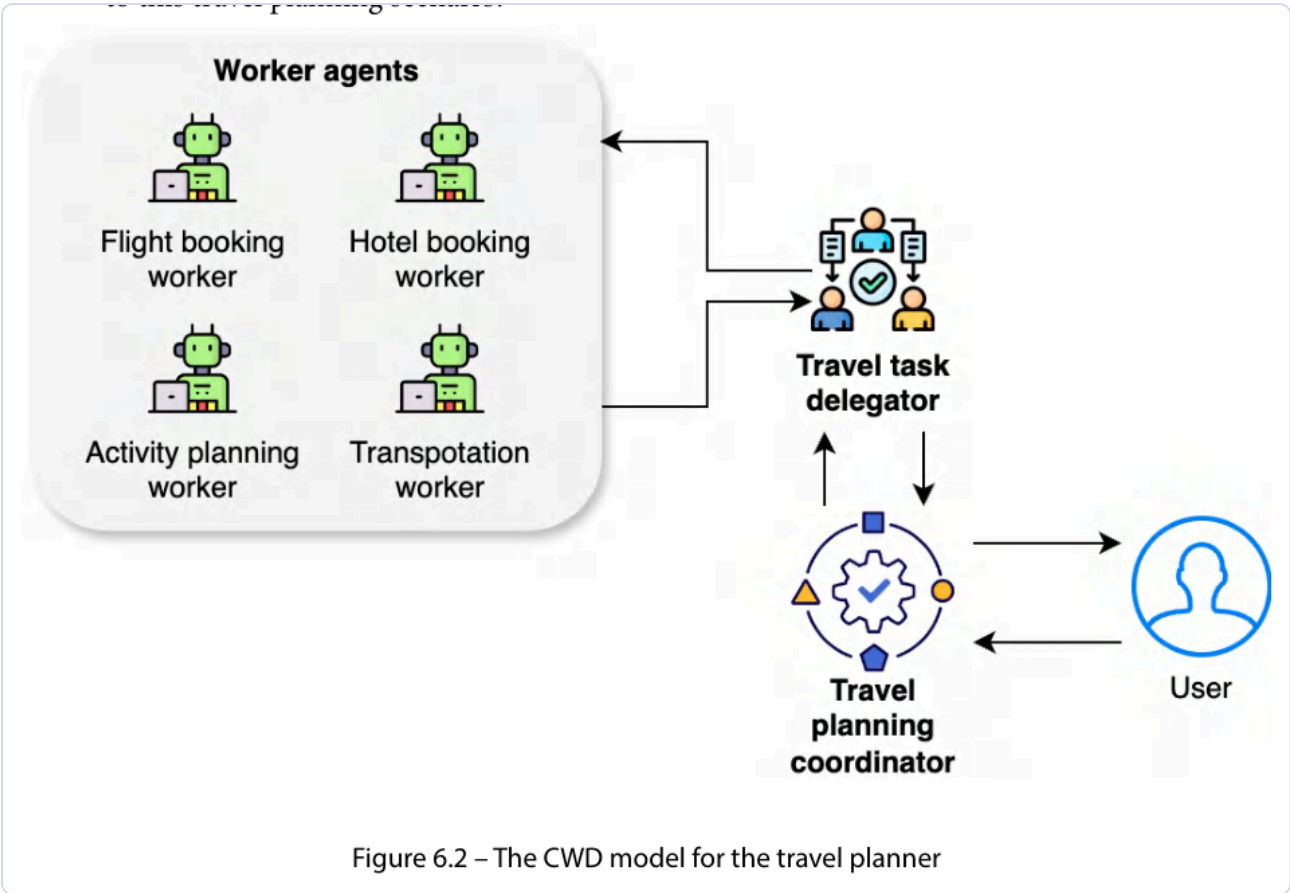


Figure 6.2 — CWD model for the travel planner

Agent	Duties
Coordinator — travel planning coordinator	Manage overall travel-planning process from user request; monitor progress & collaboration; assign tasks/coordinate workflow per customer requirements.
Workers	Flight booking (search/book by dates, destinations, preferences); Hotel booking (location, amenities, preferences); Activity planning (tours/experiences tailored to interests); Transportation (rental cars, airport transfers, local transport).
Delegator — travel task delegator	Interface between coordinator and workers; receives tasks; assesses capabilities & availability; assigns by expertise and workload; balances workload.

Workflow (9 steps)

1. User gives requirements (destination, dates, budget, preferences — family-friendly, cultural, beach).
2. Coordinator analyses and **decomposes** the task into subtasks.
3. Subtasks passed to the delegator.
4. Delegator assesses available workers and assigns tasks.
5. Workers collaborate/share information for a cohesive plan.
6. Delegator monitors progress and balances workload.
7. Workers submit outputs (flight bookings, hotel reservations, activity itineraries, transport arrangements) to the delegator.
8. Delegator compiles/integrates outputs into a comprehensive plan.
9. Coordinator reviews, adjusts and presents the plan for customer approval.

Benefits: complex tasks broken down, clear communication and responsibility, multiple requests handled simultaneously with quality. (Full code in `Chapter_06.ipynb` using CrewAI and AutoGen.)

9. Designing Agents with Role Assignments

In CrewAI each agent is initialised with **role**, **goal** and **backstory**.

- **Role** — foundation of behaviour/responsibilities; defines what the agent does and how it fits (e.g., coordinator = "Strategic Planning Manager").
- **Backstory** — a crafted narrative providing depth and context that shapes decision-making and interaction style (e.g., "veteran project manager who led diverse teams in Silicon Valley startups ..."). It is **CrewAI-specific**: CrewAI merges backstory with role into the LLM's **system prompt**.

```
coordinator = Agent(role="Strategic Planning Manager",
    backstory="A veteran project manager who has successfully led diverse teams in Silicon Valley startups,
    "
    "known for balancing innovation with practical execution. Expertise in bridging communication
    "
    "gaps between technical and non-technical teams while maintaining focus on key
    deliverables.",
    verbose=True)
```

9.1 Typical roles in CWD systems

Role	CWD mapping	Travel-agent duties
Manager	= Coordinator	Monitor planning progress; allocate resources (compute, external API access); ensure time constraints are met.
Analyst	Worker	Analyse customer preferences & trends; recommend destinations/activities; find cost savings & optimal routes; summarise outcomes.
Reflector	Worker	Observe performance; analyse feedback & satisfaction; identify bottlenecks/inefficiencies; propose improvements to algorithms/workflows.
Searcher	Worker	Explore the problem space; discover new destinations/activities; explore alternative transport/routes; share trends & regulations (innovation).
Task Interpreter	= Delegator	Map high-level tasks into concrete actions; break request into subtasks; translate preferences into tasks; ensure tasks are clear/unambiguous.

Background research: Kazík (2010) studied **role-based approaches** in multi-agent systems — roles are abstract representations of stereotypical behaviour providing interfaces through which agents get knowledge of/affect their environment. Principles carried to LLM agents: **separation of interaction logic from inner algorithmic logic, dynamic role assignment, modular system organisation** → modular, reusable designs.

9.2 Roles and responsibilities in the intelligent travel planning MAS

- **Travel planning agent (coordinator)** — "Travel Planning Executive": strategic overseer; 15 years' luxury-travel experience; goals: cohesive plans, high customer satisfaction, optimised resource allocation; handles contingencies.
- **Core travel workers:**
 - **Flight booking worker** — "Aviation Booking Specialist" (airline reservations, fare classes, routing rules, alliances); goals: optimal arrangements, max value for money, booking accuracy.
 - **Hotel booking worker** — "Hospitality Accommodation Expert" (categories, room types, loyalty programs, seasonal pricing, promotions); goals: perfect matches, best rates, special requests.
 - **Activity planning worker** — "Destination Experience Curator" (cultural knowledge + tour operations); goals: engaging itineraries, balance activities/free time, cultural authenticity.
 - **Transportation worker** — "Ground Transportation Logistics Specialist"; goals: reliable transfers, optimise local transport, maintain backup options.

- **Analysis & intelligence workers:**
 - **Travel data analyst** — "Travel Intelligence Specialist": booking patterns, preferences, market trends; goals: actionable insights, trends, customer matching.
 - **Travel experience worker (Reflector)** — "Travel Experience Optimization Expert": QA and continuous improvement; goals: analyse feedback, identify improvement areas, enhance service quality.
 - **Travel opportunity worker (Searcher)** — "Travel Discovery Specialist": explorer/innovator; goals: unique opportunities, emerging destinations, expanded offerings.
- **Delegator agent** — "Travel Operations Orchestrator": task prioritisation & resource allocation; knows worker capabilities and workload; goals: optimise task distribution, workflow efficiency, quality standards.

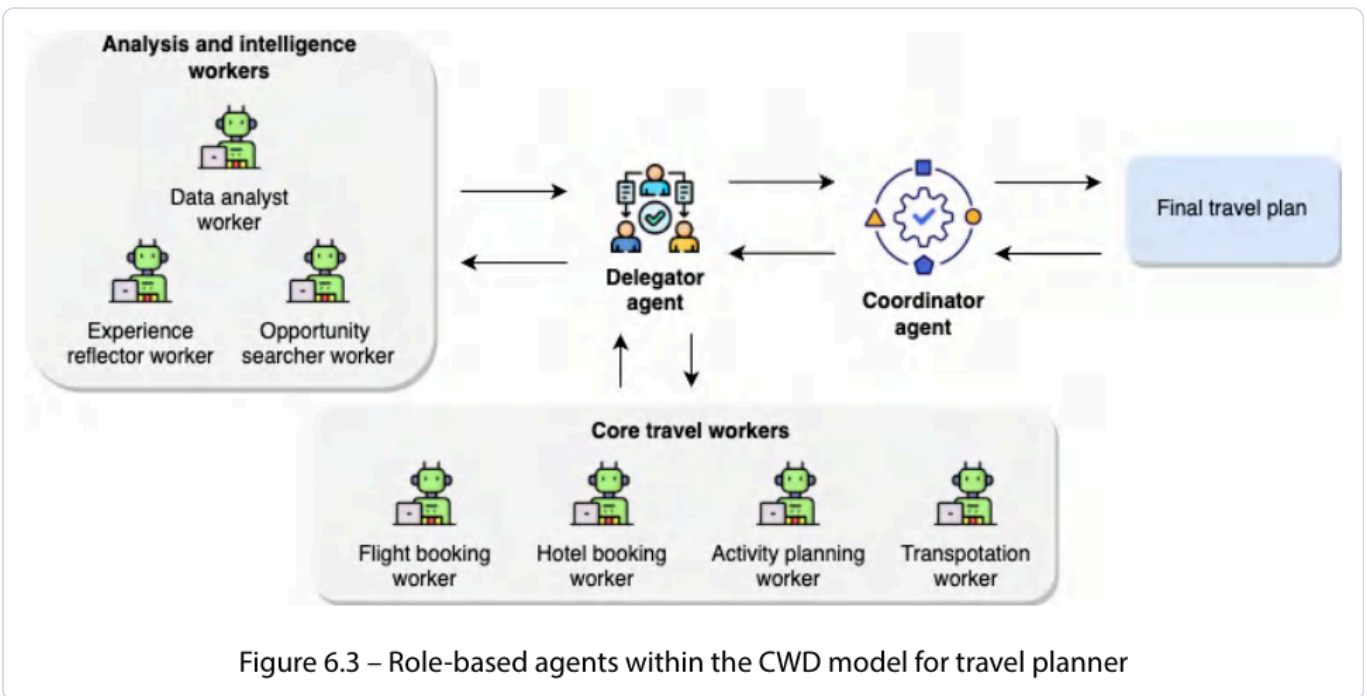


Figure 6.3 — Role-based agents within the CWD model for the travel planner

Orchestration flow

1. **Initial request & planning:** customer submits requirements; coordinator analyses & develops a strategic plan.
2. **Task distribution:** coordinator → delegator; delegator breaks the plan into tasks for core and analysis workers.
3. **Parallel processing (core tasks):** flight worker searches/reserves flights; hotel worker books accommodation; activity worker creates itinerary; transportation worker arranges ground transport.
4. **Parallel processing (analysis & intelligence):** data analyst processes customer data/patterns; experience reflector reviews similar past itineraries; opportunity searcher finds unique options.
5. **Integration & refinement:** workers submit to delegator → consolidates → coordinator receives the consolidated plan.
6. **Final review & delivery:** coordinator reviews/optimises and presents to the customer.

10. Communication and Collaboration between Agents

Aspect	Description & travel example
Communication	Well-defined protocols (message formats and interaction patterns), e.g., standardised FIPA Agent Communication Language (ACL) — message states content (requested travel dates), sender (hotel booking worker), recipient (flight booking worker).
Coordination mechanism	Coordinators align worker activities with objectives; control dependencies; ensure timeframes: task prioritisation, resource allocation, progress monitoring . E.g., if the customer prioritises accommodation, coordinator instructs delegator to give the hotel worker higher priority; reallocates resources as needed.
Negotiation & conflict resolution	When goals/actions conflict, use negotiation strategies — compromise or mediation by predefined criteria/coordinator. E.g., activity worker schedules a full-day tour but transportation worker books a rental car for the whole day: delegator mediates (reschedule tour/modify car); if unresolved, coordinator decides using predefined criteria or customer preference.
Knowledge sharing	Agents share insights for continuous learning/adaptation. E.g., data analyst spots a rising eco-friendly travel trend → shares with activity & transportation workers; searcher finds a new eco-resort → shares with hotel & activity workers.

The role-based approach sets clear boundaries for communication channels and responsibilities; protocols for communication, coordination and knowledge sharing harness collective intelligence.

11. Implementing CWD in Generative-AI Systems (introduction)

- In **traditional MAS** behaviours/interactions are explicit code; in **LLM-based systems** they are controlled by **carefully crafted prompts and interaction patterns** — must preserve clear role boundaries and hierarchy.
- Three key implementation considerations: **system prompts, instruction formatting, interaction patterns**.
- **System prompts** = fundamental configuration layer: define persistent traits, boundaries and behavioural frameworks over the agent's life cycle (unlike task-specific prompts). Each agent's prompt must include: **role definition & scope of responsibilities; constraints & operational boundaries**; (continued in Unit 4) communication protocols and decision-making frameworks.
- Later chapter topics previewed: state-space management, environment modelling, memory systems, handling LLM contexts.

12. Unit Summary

- Practical planners for LLM agents: **LLM-based** and **HTN**; FF moderately; STRIPS/A*/GraphPlan/MCTS impractical.
- Tools + planning together ground plans in facts (without tools the plan is invented).
- Frameworks: **CrewAI** (role/goal/backstory, sequential or hierarchical, manager), **AutoGen** (round-robin group chat, termination keyword), **LangGraph** (state graph with agent & tool nodes, checkpointing).
- **CWD** = Coordinator (strategy) → Delegator (resource mgmt/dispatch) → Workers (execution); principles: separation of concerns, hierarchy, bidirectional flow, adaptability/resilience.
- Role design: role + goal + backstory form the system prompt; Manager=Coordinator, Task Interpreter=Delegator, Analyst/Reflector/Searcher=Workers.
- Collaboration needs: communication protocols (FIPA ACL), coordination, negotiation/conflict resolution, knowledge sharing.

Likely exam questions 1) Explain LLM-based planning and HTN with a travel example. 2) Compare CrewAI, AutoGen and LangGraph. 3) Explain the CWD model, its roles and principles with a diagram. 4) Describe the travel planner's role-based agents and orchestration flow. 5) How do agents communicate, coordinate, negotiate and share knowledge?

Sources: Unit-3 Part-1 (Enabling Tool Use and Planning in Agents, Ch.5 cont.) & Unit-3 Part-2 (Coordinator, Worker, and Delegator Approach, Ch.6) — summarised for RVCE study use.